1. Introduction

This assignment implements a stack-based mathematical expression evaluator in Python. The program reads multiple mathematical expressions from input.txt, evaluates each expression using a custom stack data structure, and writes the results to output.txt while preserving the separator structure used in the input file. The required arithmetic operators are addition (+), subtraction (-), multiplication (*), and division (/), together with parentheses and standard operator precedence.

The solution deliberately avoids Python's eval() function. Instead, it demonstrates how stacks can be used to store operands and operators while an expression is processed. This directly addresses the learning objective of applying a fundamental data structure to a practical computing problem.

2. Data Structures Used

2.1 Stack Data Structure

A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle. The most recently inserted item is the first item removed. The implementation in this project provides push(), pop(), peek(), is_empty(), and size() operations. Internally, a Python list is used to store the stack items, but all expression processing is performed through the Stack class.

2.2 Use of Stacks in Expression Evaluation

Two stacks are used. The value stack stores numeric operands and intermediate results. The operator stack stores arithmetic operators and parentheses. When an operator with lower or equal precedence is encountered, higher- or equal-precedence operators already on the operator stack are applied first. Parentheses temporarily control this process so that expressions inside parentheses are evaluated before surrounding operations.

2.3 Importance of Stacks in Computing

Stacks are important in computing because many problems naturally require LIFO processing. They are used in function-call management, recursion, undo operations, parsing, syntax checking, expression evaluation, depth-first search, and compiler/interpreter processing. Expression evaluation is a clear example because operators must be temporarily stored until their precedence and operand requirements allow them to be applied.

3. Program Design

3.1 Program Architecture

The program is organized into a small set of focused components. The Stack class provides the required data-structure operations. tokenize() converts an expression into valid tokens. precedence() assigns precedence levels to operators. apply_operator() performs an arithmetic operation using two operands. evaluate_expression() coordinates the two stacks and applies the precedence and parenthesis rules. format_result() produces clean output, while process_file() reads input.txt, evaluates each expression, preserves separators, and writes output.txt.

3.2 Key Functions

Function	Purpose
Stack.push(item)	Adds an item to the top of the stack.
Stack.pop()	Removes and returns the top item; raises an error if the stack is empty.
Stack.peek()	Returns the top item without removing it.
tokenize(expression)	Breaks an expression into numbers, operators, and parentheses.
precedence(operator)	Returns 1 for +/− and 2 for */.
apply_operator(values, operator)	Pops two operands, applies the selected operator, and pushes the result.
evaluate_expression(expression)	Evaluates the complete infix expression using the value and operator stacks.
process_file(input_file, output_file)	Reads expressions, evaluates them, preserves separators, and writes results.

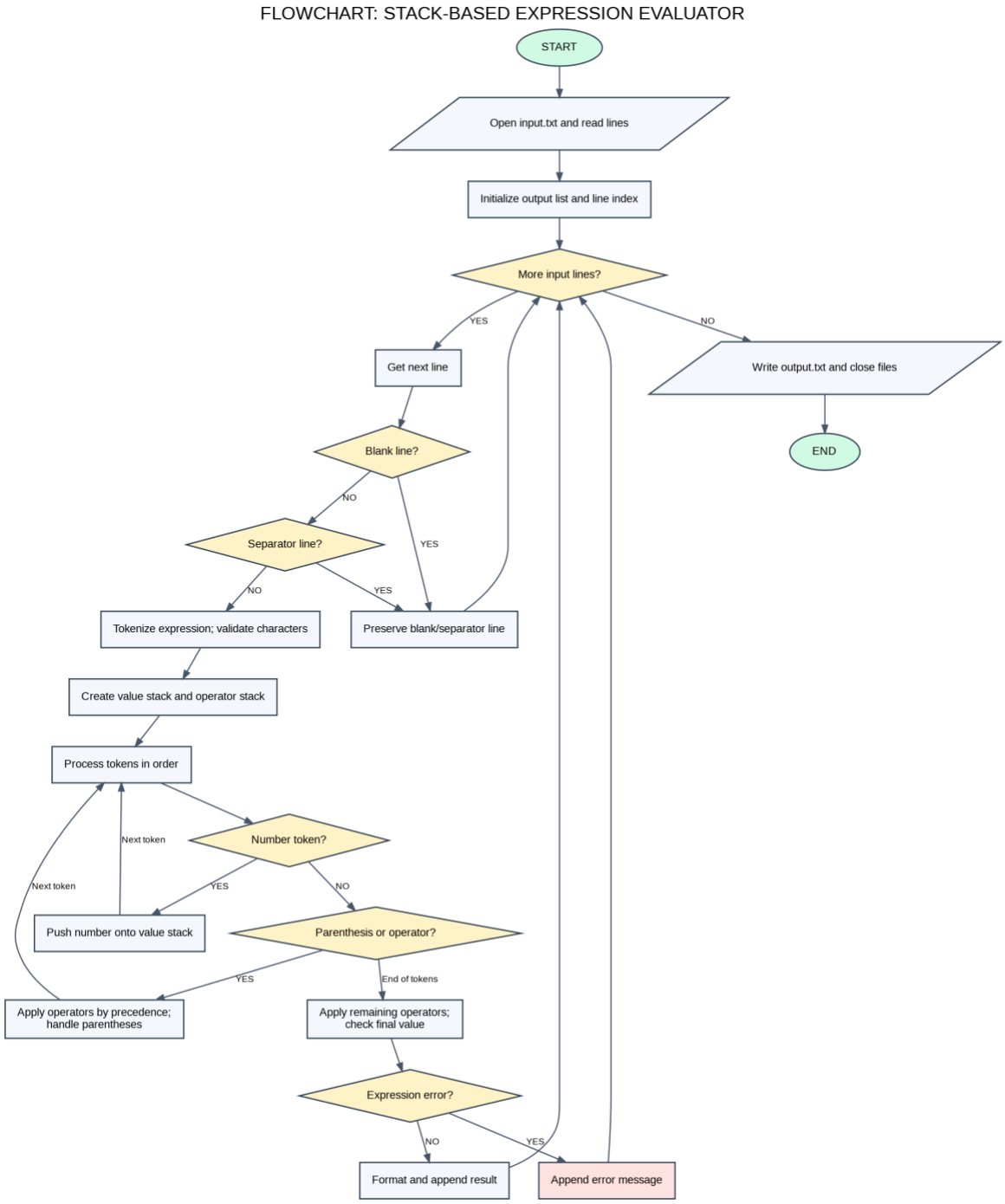
3.3 Pseudocode

```
START
    Read all lines from input.txt
    FOR each line
        IF line is a separator
            copy separator to output
        ELSE
            tokenize expression
            create empty value stack
            create empty operator stack

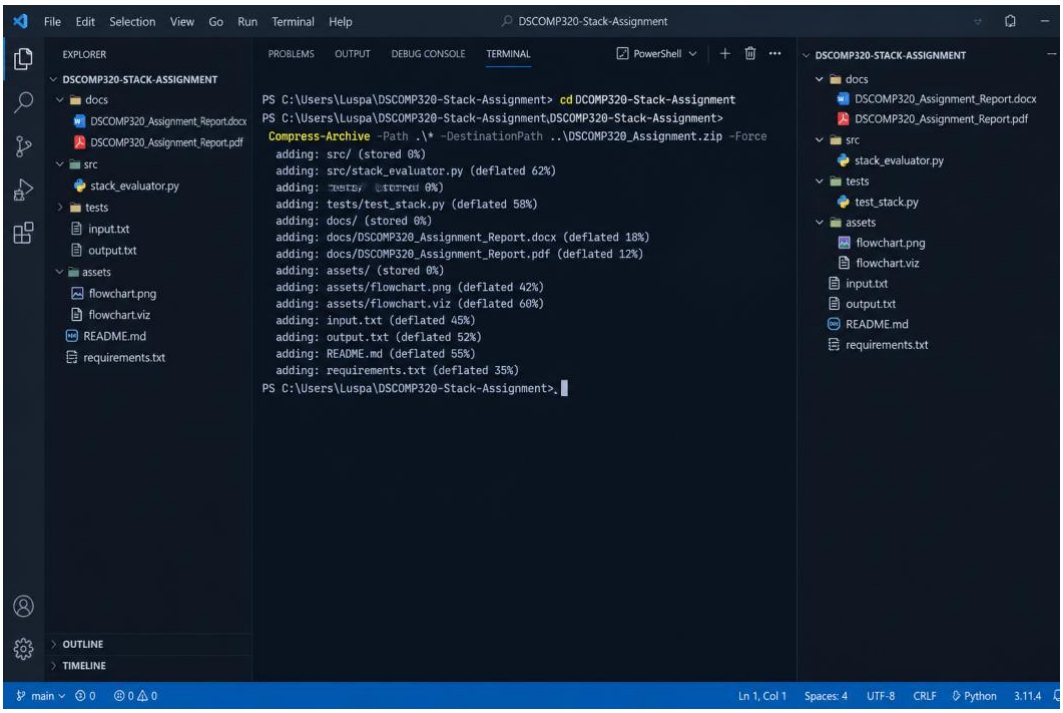
            FOR each token
                IF token is a number
                    push number onto value stack
                ELSE IF token is "("
                    push it onto operator stack
                ELSE IF token is ")"
                    apply operators until "(" is found
                    remove "("
                ELSE IF token is an operator
                    apply higher/equal precedence
operators
                    push current operator

            apply remaining operators
            take final value from value stack
            write result to output
    Write output.txt
END
```

3.4 Flowchart



4. Testing and Results



4.1 Test Input

The supplied sample structure was used for testing. The input file contains three expressions, with a separator line between expressions:

```
3 + 5 * 2
-----
(8 / 4) + 7 * 2
-----
10 - (2 + 3) * 4
```

4.2 Test Output

```
13
-----
16
-----
-10
```

The first expression confirms operator precedence because multiplication is performed before addition: 3 + (5 × 2) = 13. The second expression confirms parentheses and precedence: (8 ÷ 4) + (7 × 2) = 2 + 14 = 16. The third expression confirms that the parenthesized subtraction is performed first and the multiplication is then applied: 10 − ((2 + 3) × 4) = −10.

A discrepancy is visible in the scanned assignment brief: its example image appears to show 17 for the second expression. Under the arithmetic expression printed in the brief, the correct result is 16. The program therefore follows the mathematical rules explicitly requested by the assignment rather than reproducing the apparent example typo.

4.3 Error Handling Tests

Additional checks were built into the evaluator. Invalid characters, insufficient operands, mismatched parentheses, unsupported operator placement, and division by zero are detected. An error is written for the affected expression while processing can continue for subsequent lines.

5. Conclusion

The completed program demonstrates the practical use of the stack data structure for evaluating infix mathematical expressions. It includes the required stack operations, input-file processing, expression evaluation, output-file generation, precedence handling, parenthesis handling, and basic error handling. The project also includes a README with execution instructions and a sample input/output pair suitable for a GitHub repository.

For the GitHub submission requirement, the completed project should be pushed to a repository and the lecturer should be added as a collaborator through the repository settings.

6. References

• Limkokwing University of Creative Technology, DCOMP320 Data Structures and Algorithm – Individual Assignment: Stacks Implementation, September 2026.

• Python Software Foundation, Python 3 Documentation – Built-in Types and File I/O.

• Python Software Foundation, Python 3 Documentation – re (Regular Expression Operations).